

JavaScript Notes: Rest & Default Parameters, Array Methods, and Classes

Daniyal

oday

1. Rest Parameters

The rest parameter syntax (`...name`) lets a function accept an indefinite number of arguments as a **real array**. It must be the **last** parameter in the function definition, and there can only be one.

```
function test(a, b, ...c) {  
  console.log(a, b, c);  
}  
  
test(1, 2, 3, 4, 5);  
// a = 1  
// b = 2  
// c = [3, 4, 5] <-- everything else is collected into an array
```

Key points

- `c` is a genuine array, so you can call `.map()`, `.filter()`, `.reduce()`, etc. on it directly.
- If no extra arguments are passed, the rest parameter becomes an empty array `[]`, not `undefined`.
- Different from the old `arguments` object, which is array-like, not a real array.

Extra example

```
function sum(...numbers) {  
  return numbers.reduce((total, n) => total + n, 0);  
}  
  
console.log(sum(1, 2, 3)); // 6  
console.log(sum(10, 20, 30, 40)); // 100
```

2. Default Parameters

Default parameters let you assign a fallback value to a parameter, which is used **only when the argument is undefined** (not passed, or explicitly passed as `undefined`).

```
function test2(x = 5, y, z = 5) {  
  console.log(x, y, z);  
}  
  
test2(1);  
// x = 1 (explicitly passed, overrides default)  
// y = undefined (no default given, and nothing passed)  
// z = 5 (nothing passed, so default is used)
```

Key points

- Passing `null` does **not** trigger the default — only `undefined` does.
- Default values can reference earlier parameters:

```
function greet(name = "Guest", greeting = `Hello, ${name}!`) {  
  console.log(greeting);  
}  
  
greet(); // Hello, Guest!
```

```
greet("Daniyal"); // Hello, Daniyal!
```

3. Array Methods: `map`, `filter`, `reduce`

These are higher-order functions that operate on arrays *without* mutating the original array (they return a new array or value instead).

3.1 The manual (old-school) way

```
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const arr2 = [];
for (let i = 0; i < arr.length; i++)
  arr2.push(arr[i] * 2);
console.log(arr2); // [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

This is exactly what `map()` does for you, but with a manual loop.

3.2 `map()` — transform every element

`map()` runs a function on every element and returns a **new array of the same length**, where each element is the result of the function.

```
let arr3 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
arr3 = arr3.map((v) => v * 2);
console.log(arr3); // [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

Use `map` when you want to transform each item and keep the same number of items.

3.3 `filter()` — keep only what matches

`filter()` runs a test function on every element and returns a **new array containing only the elements that passed** (the function returned `true`).

```
let arr4 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
arr4 = arr4.filter((a) => a === 2);
console.log(arr4); // [2]
```

Note: `filter` always returns an array, even if only one item (or zero items) matches.

Extra example

```
const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const evens = nums.filter((n) => n % 2 === 0);
console.log(evens); // [2, 4, 6, 8, 10]
```

3.4 `reduce()` — collapse the array into one value

`reduce()` runs a function on each element that carries an **accumulator** forward, ultimately “reducing” the array to a single value (a sum, an object, a string, etc.).

```
let arr5 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
arr5 = arr5.reduce((p, c) => {
  return p + c;
}, 0);
console.log(arr5); // 55
```

- `p` = accumulator (previous result)
- `c` = current array element

- `0` = the starting value of the accumulator (passed as the second argument to `reduce`)

Extra example — finding the maximum

```
const scores = [45, 92, 67, 88, 71];
const highest = scores.reduce((max, current) => {
  return current > max ? current : max;
}, scores[0]);
console.log(highest); // 92
```

3.5 Quick comparison

Method	Returns	Use case
<code>map</code>	New array, same length	Transform each item
<code>filter</code>	New array, shorter/equal length	Keep items that pass a test
<code>reduce</code>	Single value	Combine all items into one result

4. Classes

Classes are syntax sugar over JavaScript's prototype-based inheritance, giving a cleaner way to create objects with shared structure and behavior.

```
class Person {
  constructor(name, id) {
    this.name = name;
    this.id = id;
  }

  greet() {
    console.log(`hello from ${this.name}`);
  }

  static test() {
    console.log(`test from class`);
  }

  get getId() {
    return this.id;
  }

  set setId(data) {
    this.id = data;
  }
}
```

4.1 constructor

Runs automatically when a new object is created with `new` . It sets up the instance's initial properties.

```
const person1 = new Person("Daniyal", 26);
const person2 = new Person("Faizan", 25);

console.log(person1); // Person { name: 'Daniyal', id: 26 }
console.log(person2); // Person { name: 'Faizan', id: 25 }
```

4.2 Instance methods

Regular methods (like `greet`) live on the class's prototype and are shared by every instance, but operate on `this` — the specific object calling them.

```
person2.greet(); // hello from Faizan
```

4.3 Static methods

static methods belong to the **class itself**, not to instances. You call them on the class name directly, and they cannot access `this.name` or other instance data.

```
Person.test(); // test from class
// person1.test(); // ERROR: not available on instances
```

Static methods are useful for utility/helper functions related to the class (e.g. `Person.compare(a, b)`).

4.4 Getters and setters

Getters and setters let you access/update a property using normal property syntax (`obj.prop`), while still running custom code behind the scenes — like validation or computed values.

```
person1.setId = 29; // calls the setter, id becomes 29
console.log(person1.getId); // calls the getter, prints 29
```

Note: Even though `setId` and `getId` are defined as methods, you use them **without parentheses**, because `get` / `set` make them behave like properties, not function calls.

Extra example — validation inside a setter

```
class Product {
  constructor(price) {
    this._price = price;
  }

  get price() {
    return this._price;
  }

  set price(value) {
    if (value < 0) {
      console.log("Price cannot be negative");
      return;
    }
    this._price = value;
  }
}

const p = new Product(100);
p.price = -50; // "Price cannot be negative" - rejected
console.log(p.price); // 100
```

5. Summary

- **Rest parameters** (`...args`) collect extra arguments into a real array — must be last, only one allowed.
- **Default parameters** fill in a value only when an argument is `undefined`.
- **map** transforms each element into a new array of the same length.
- **filter** keeps only elements that pass a test.
- **reduce** folds the whole array into a single accumulated value.
- **Classes** bundle a **constructor**, instance methods, **static** methods (called on the class, not instances), and **get** / **set** accessors (used like properties, not called like functions).